

Lecția 12 – Python

Trainer: Hrișcă Miruna

Aplicații folosite + alternative ▾

01

PyCharm

[Link - windows](#)

[Link - Apple](#)

02

One Compiler

[Link](#)

03

VS Code

[Link](#)

04

Online GDB

[Link](#)

Biblioteca Time și OS



❖ Time



- Python are un modul numit **time**.
- Acesta ne ajută să facem lucruri legate de **time** în program.
- De exemplu, putem spune programului să aștepte câteva secunde înainte să continue.
- **Funcția sleep()** - Întârzie execuția programului pentru un număr de secunde.



❖ OS



- Modulul **os** ne ajută să facem lucruri legate de **calculator și sistemul de operare**.
- Este preinstalat, deci nu trebuie să îl instalăm.



Proiect Rachetă



```
import time
import os
```



- `import time` → aduce în program **biblioteca `time`**, care ne permite să lucrăm cu timpul (ex: să punem pauze între acțiuni).
- `import os` → aduce **biblioteca `os`**, care permite să facem lucruri legate de sistemul de operare, cum ar fi **ștergerea ecranului**.

```
run_length = 50
```



- Aceasta spune **cât de sus vrem să „pornim” desenul în terminal**.
- Este practic **numărul de linii goale** înainte de desen.

👁👁 Important: Dacă `run_length` scade, desenul „coboară” pe ecran.

```
^
|
|
|
|/|\
```

Proiect Rachetă

```
while True:
```

- `while True:` înseamnă **repetă la nesfârșit** tot ce e sub această linie.
- Este o buclă infinită, deci trebuie să avem grijă să avem **condiții care să schimbe ceva** ca să nu blocăm programul.

```
print('\n' * run_length)
```

- `'\n'` este o linie nouă.
- `'\n' * run_length` înseamnă „linie nouă, de `run_length` ori” → face ca desenul să apară mai jos pe ecran.

```
^
|
|
|
|/|\
```

Proiect Rachetă



```
print('\t\t\t /\t\t')
print('\t\t\t ||')
print('\t\t\t ||')
print('\t\t\t /||\t')
```

- `\t` → tab, adică un spațiu mare, ca să **aliniem desenul**.
- Acestea formează un **mic copac sau steag**.

⚡ Important: `\t` și `\n` sunt caractere speciale pentru spațiu și linie nouă. Ele fac ca desenul să fie „frumos” aranjat.



```
time.sleep(0.05)
```

- `time.sleep(secunde)` oprește programul pentru **0.05 secunde**.
- Aceasta face ca desenul să pară că se mișcă, ca o **animație**.

👁 Important: fără `sleep`, terminalul ar afișa totul instant și nu am vedea mișcarea.

```
 /\t\t
 ||
 ||
 ||
 /||\t
```



- `run_length -= 1` → scade numărul de linii goale, adică desenul urcă pe ecran.
- `if run_length < 0`: → când ajunge sus, **resetăm** la 50, ca desenul să reînceapă de jos.

⚡ Important: Aceasta face animația **circulară**, racheta „urcă” și apoi reapare jos.



Proiect Numărătoare inversă

```
import time
```

- În acest cod, o folosim ca să **așteptăm câte o secundă între numere**, pentru ca timer-ul să meargă ca unul real.

```
timp= int(input("Introduceti timpul in secunde: "))
```

- `input()` cere utilizatorului să scrie ceva.
- `int()` transformă ceea ce scrie utilizatorul într-un **număr întreg**.
- Rezultatul îl punem în variabila `timp`, adică **cât vrem să numărăm**.

👁 Important: `input()` returnează întotdeauna text, de aceea folosim `int()`.

Proiect Numărătoare inversă

```
for x in range(timp,0,-1):
```

- `range(start, stop, step)` creează o listă de numere.
 - `start = timp` → începem de la timpul introdus.
 - `stop = 0` → mergem până la 1 (0 este exclus).
 - `step = -1` → scădem câte 1 la fiecare pas, adică numărăm invers.
- ⚡ Important: această buclă scade automat și se repetă pentru fiecare secundă.

Proiect Numărătoare inversă

```
secunde = x % 60
minute = int(x / 60) % 60
ore = int(x / 3600) % 24
```

- $x \% 60$ → restul împărțirii la 60, adică **câte secunde rămân** după ce facem minutele.
- $\text{int}(x / 60) \% 60$ → cât este **numărul de minute** (excluzând orele complete).
- $\text{int}(x / 3600) \% 24$ → cât este **numărul de ore** (excluzând zilele).

⚡ Important: $\%$ (modulo) ia **restul împărțirii**, iar $\text{int}(x / 60)$ ia doar partea întreagă a împărțirii.

👁️ Astfel, dacă introducem 3661 secunde, programul va afișa 01:01:01.

Proiect Numărătoare inversă

```
print(f"{ore:02}:{minute:02}:{secunde:02}")  
print(x)
```



- `f"{ore:02}"` → afișează orele **cu două cifre**, chiar dacă sunt mai mici de 10 (ex: 01, 05).
- `print(x)` → afișează timpul rămas în secunde (mai mult pentru verificare).

⚡ Important: formatul `02` face ca timer-ul să arate frumos, ca un ceas real.

```
time.sleep(1)
```

- Oprește programul pentru **1 secundă**, ca să pară că timer-ul scade real.

```
print("TIMP SFARSIT!")
```

- Când bucla se termină, programul afișează că **timpul s-a terminat**.



Aplicație extra



Aplicația 1

Rezolvă câteva nivele din jocul de mai jos indicate de trainer:

<https://codecombat.com/>

